Certificate-Gated Neural Proposers for Verifier-Grounded Action Scoring

PLACEHOLDER: Authors and affiliations

Working draft — ideal-paper roadmap

Abstract. When an exact verifier is available, the right architecture for action scoring is not an end-to-end neural policy. We specify one reusable verifier-gated action task schema: complete candidate enumeration, observable state/action features, a neural residual scorer, an exact verifier, a symbolic rule grammar, a certificate checker, a rule proposer, and a trace/replay/RL protocol. We instantiate this schema on three tasks: prime gaps, grid paths, and delayed claims. Prime gaps currently provide the strongest gate ablation: with residue-sieve features removed from the scorer input, the raw scorer reaches 9.13% top-1; a fixed hand-wheel gate raises this to 45.12%, and a learned rule gate reaches 68.87% at 100% top-10 with an ≈4× reduction in model latency. Grid paths show the same certified-gate pattern outside arithmetic at smaller scope: with Manhattan priors disabled, a budgeted legality gate raises grid-path accuracy from 70.80% to 84.90%; the priors-allowed checkpoint is retained as an explicit ablation at 75.80% raw and 89.60% gated. Delayed claims currently exercise trace replay and calibration, not gate learning. The prime rule-search negative result is load-bearing: grammar order and a simple structural heuristic beat the current neural proposer at a fixed 16-rule budget, and the conditional gap grammar exposes a larger failure. The contribution is therefore the generalized protocol and audit discipline, not an improved prime algorithm.

1. Introduction

Exact verifiers are common in domains where action correctness can be checked algorithmically even when discovery cannot: theorem proving, program synthesis, combinatorial search, formal shortest-path problems. The dominant style in modern learned policy work treats the verifier as a reward signal for an end-to-end neural policy. We argue this is the wrong default when the candidate space contains many structurally impossible actions that the verifier can cheaply eliminate.

Our thesis is the **certificate-gating loop**: a learned scorer should consume the **survivors** of a verifier-checked symbolic filter, and the filter itself can be produced by a rule proposer gated by a soundness-certificate verifier. The scorer ranks the residual; the gate eliminates the rest; nothing the gate emits can violate correctness because every rule it accepts carries a deterministic proof.

We make three protocol claims, each of which must be instantiated across tasks before being treated as general:

(C1) **Certified gates reduce the residual action space.** The complete prime-gap result and the smaller grid-path legality result support this. Delayed-claim gate learning remains to be implemented.

(C2) **Rule search must beat fixed-budget symbolic baselines.** The current prime result is negative for neural proposer learning: grammar order beats the neural proposer on the unconditional candidate grammar, and the conditional gap grammar makes the failure larger. This turns proposer learning into a concrete cross-task benchmark rather than a tautology.

(C3) **Verifier RL must preserve calibration across tasks.** Strict policy gradient on sparse exact rewards inflates logit scale on prime gaps; a standard KL-to-prior + entropy term restores calibration at unchanged top-k. The same run is still needed on grid paths and delayed claims.

The artifact now emits `crlp.experiment_schema.v1` rows for every task with `make experiment-schemas`. The paper should be read against that schema: task-specific verifiers and grammars change, but candidate enumeration, leakage discipline, proposer baselines, trace replay, and verifier-RL comparisons are shared.

2. Related Work

PLACEHOLDER — Required citations and 1-paragraph positioning against each:

- Process supervision (Lightman et al., 2023): we use a similar verifier-as-step-checker pattern; their reward model is learned, ours is exact-by-construction.
- Verifier-guided decoding / best-of-N (Cobbe et al., 2021; Hosseini et al., 2024): we gate **before** scoring, not after.
- AlphaProof / AlphaGeometry symbolic-neural search (Trinh et al., 2024): our rule synthesis is structurally similar but verifier-grounded by certificate rather than by proof completion.
- RLHF KL-regularized policy gradient (Stiennon et al., 2020; Ouyang et al., 2022): Axis 3 applies this objective to exact-verifier rewards.
- Neural-symbolic program synthesis (NeSy literature).

3. Method

3.1. The Certificate-Gating Loop

The reusable architecture is:

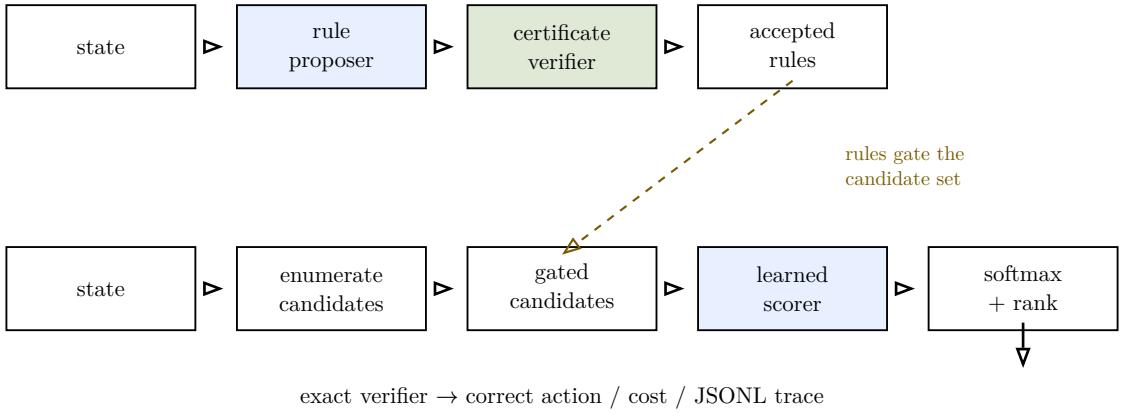


Figure 1: The certificate-gating loop. A rule proposer (top) produces candidate rules that a soundness-certificate verifier accepts or rejects. Accepted rules deterministically prune the candidate set; a shared neural scorer ranks the survivors; the exact verifier supplies the correctness signal.

The design invariant is **the model proposes, the verifier decides, the trace remembers.** A rule r is accepted only if a deterministic algebraic check certifies that r cannot reject the true action on any state in the support set, so soundness does not depend on the proposer being well-trained.

3.2. Generalized Task Schema

Every experiment in the artifact emits the same schema row:

```
Task T = {
  state distribution S_T,
  candidate enumerator A_T(s),
  observable features x_T(s, a),
  neural residual scorer f_theta(x_T),
  exact verifier V_T(s, a),
  symbolic grammar G_T,
  certificate checker C_T(rule),
  rule proposer q_phi(rule | structural(rule)),
  gate executor R_T(rule, s, a),
  trace/replay/RL protocol
}
```

Task	Actions	Verifier	Certificate
`prime_gap`	candidate gaps	segmented sieve next-prime verifier	algebraic divisibility proof
`grid_path`	four moves	BFS shortest-path successor	cannot reject an optimal move
`delayed_claim`	trust / ignore / request / cautious	deterministic hidden-outcome simulator	finite-generator safety check

Table 1: The three task instantiations share the same verifier-gated action-scoring contract.

The leakage rule is shared: verifier outputs may be labels, rewards, certificates, diagnostics, or executed gates, but they are not default scorer inputs. Proposer inputs are structural rule descriptors; support-derived marginal utility labels and false-rejection diagnostics are targets, not features. In the artifact this is a typed contract, not just prose: every scorer feature declares provenance (`state_observation`, `structural_prior`, `verifier_derived`, or `derived_from_gate`), every task declares a leakage policy (`forbidden`, `priors_allowed`, or `legacy`), and schema emission fails if enabled scorer features violate that policy.

3.3. Trace and Checkpoint Contract

Every step writes a JSONL row with schema `crlp.trace.v1`, including a stable per-action identifier, the candidate scores and probabilities, the selected and target indices, the verifier result, and verifier cost. Replay resolves `target_index` $\rightarrow$ stable `action_id` $\rightarrow$ regenerated candidate row, so reordering action ids cannot silently change replay semantics. Checkpoints carry SHA-256 model identity, a `parent_model_hash`, a `feature_encoder_version`, and a `feature_set`; comparison reports (`crlp.compare.v1`) carry both hashes, the evaluated `feature_set`, and machine-readable reliability bins.

4. Task Instantiation: Prime Gaps

4.1. Setup

The state is a prime p; the candidate space is gaps $g \in \{2, 4, ..., 256\}$ plus an overflow action; the verifier returns the true next-prime gap. The default candidate encoding is the minimal feature set $[\log p, \text{norm_gap}, \text{overflow_flag}]$. A full-sieve feature set that adds precomputed divisibility flags for $p + g$ modulo small primes is retained only as a leakage ablation. The scorer is a shared MLP returning one scalar per candidate; softmax over candidates produces the proposal distribution.

The default split is $p < 800,000$ train, $800,000 \leq p < 1,000,000$ validation, $1,000,000 \leq p \leq 3,000,000$ test; all reported numbers use 30,000 held-out test states unless stated otherwise.

4.2. Why this domain

- A fast deterministic verifier at all scales (segmented sieves, wheel filters, block proofs).
- A compact totally-ordered candidate space.
- An exhausted classical literature — we cannot beat a sieve, which sharpens the question to "what does the learned component contribute under perfect feedback?"
- Rich algebraic structure for the rule grammar (residue classes, divisibility certificates).

5. Axis 1: Certified gates reduce the residual action space

The same trained prime-gap scorer is evaluated under three gate conditions on the canonical 30,000-sample held-out benchmark. Top-1 results:

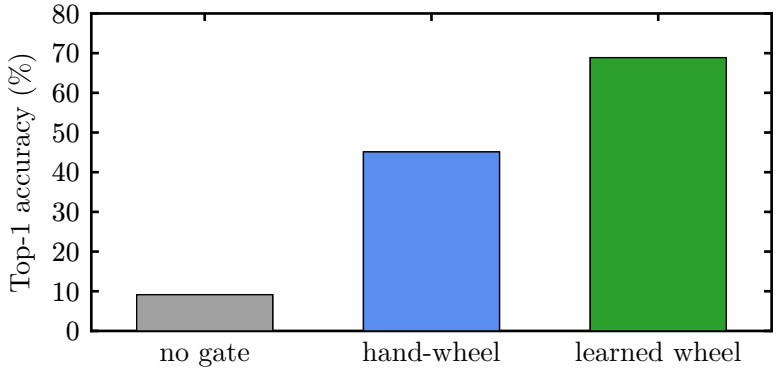


Figure 2: Top-1 accuracy by gate on the canonical 30,000-sample held-out benchmark. The clean raw-to-learned delta is **9.13 → 68.87**. The fixed-hand-wheel-to-learned delta is **45.12 → 68.87** (+23.8pp).

The former 45.12% raw-scorer result is now a full-sieve-feature leakage ablation, not the headline.

Full metrics:

Gate	Top-1	Top-10	NLL	ECE	ms/sample
no gate	9.13%	78.01%	2.9994	0.1076	0.0365
hand-wheel	45.12%	99.77%	1.5902	0.0834	0.0375
learned wheel	**68.87%**	**100.00%**	**1.0056**	0.1402	**0.0060**

Table 2: Gate ablation, minimal-feature scorer, identical weights across rows.

PLACEHOLDER — Required to elevate C1 from "load-bearing on prime-gap" to "general":

- **E1.1** — Random-rule gate at matched verifier-call budget: replace the learned rule set with k random sound rules and report the top-1 / top-10 / NLL curve as k varies. Resolves whether the **content** of the learned rules matters or only their count.
- **E1.2** — Extend the second-domain `grid_path` check beyond the current budgeted legality gate to admissible-heuristic pruning. Same scorer + three gates + same table.
- **E1.3** — Investigate the learned-wheel calibration tradeoff. The learned wheel improves ranking sharply but has higher ECE than raw minimal and hand-wheel rows in the default run.
- **E1.4** — Confidence intervals across ≥ 5 seeds for the 45.12 → 68.87 delta.

6. Axis 2: Rule search must beat fixed-budget symbolic baselines

The canonical rule grammar is `reject candidate n when n > m and n % m == r`; the certificate verifier accepts a rule iff $\gcd(m, r) > 1$ (so n is necessarily composite). The matched-budget baseline table also reports conditional gap filters (`if p % m == a then reject gap % m == b`) because that

grammar is less saturated and exposes proposer failure more clearly. Zero false rejections is a structural property of the certificate. The proposer input is structural-only: condition kind, moduli, reciprocal scales, residues, zero-residue indicators, gcd ratios, and derived candidate residue. Empirical support and false-rejection counts are diagnostics, and budgeted marginal utility is the neural target, not a proposer feature.

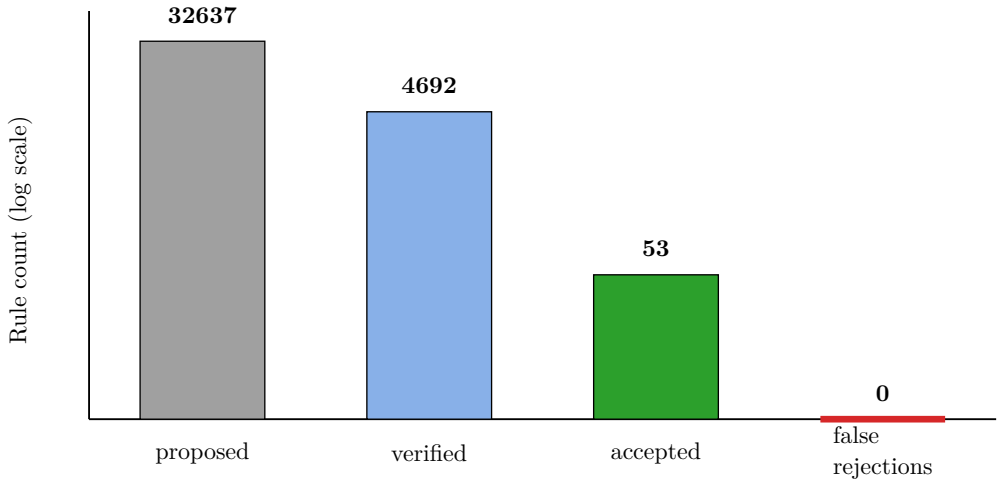


Figure 3: Rule-search acceptance funnel. Of 32,637 proposed rules, 4,692 pass the soundness certificate, 53 survive support and false-rejection screens, and **zero** falsely reject the true next-prime gap. The accepted set recovers 79.79% of wheel-equivalent verifier work.

Grammar	Rule ordering	Budget	Exact probes	Saved
candidate	neural structural	16	1.98	72.49%
candidate	random, seed 20240511	16	3.86	46.30%
candidate	grammar order	16	1.93	73.22%
candidate	structural heuristic	16	1.93	73.22%
candidate	validation-marginal oracle	16	1.93	73.22%
conditional gap	neural structural	16	7.04	2.15%
conditional gap	random, seed 20240511	16	6.94	3.49%
conditional gap	grammar order	16	3.20	55.57%
conditional gap	structural heuristic	16	3.20	55.57%
conditional gap	validation-marginal oracle	16	3.11	56.83%

Table 3: Matched 16-rule-budget proposer baselines over the same split and certificate verifier. Lower exact probes is better.

What this evidence supports. "Certificate-gated structural rule discovery over this grammar produces a sound, useful rule set without support-count feature leakage." The soundness comes from the certificate; the coverage comes from the grammar plus accepted certified filters.

What this evidence does not support. "The neural proposer, specifically, is essential." Table 3 is a negative result for that stronger claim: simple deterministic orderings beat the neural structural proposer at the fixed 16-rule budget on both reported grammars, and the conditional gap grammar exposes a larger failure. The next neurosymbolic experiment must use a richer grammar and show that learned ordering beats grammar-order and structural-heuristic baselines under the same budget.

PLACEHOLDER — Required to elevate C2 from "the verifier is doing useful work" to "the learned proposer is useful":

- **E2.1** — Repeat the fixed-budget proposer baseline across random seeds and rule budgets, reporting confidence intervals.
- **E2.2** — Generalize the grammar: extend to conjunctive rules (`if p % m == a and ... then reject ...`), report whether the proposer scales.
- **E2.3** — Show one **worked example** of an accepted rule the hand-wheel does not capture and explain why it generalizes.

7. Axis 3: Verifier RL must preserve calibration

Starting from a single supervised checkpoint, we clone two RL branches: strict policy gradient on exact rewards, and the same gradient regularized with KL-to-supervised-prior + entropy.

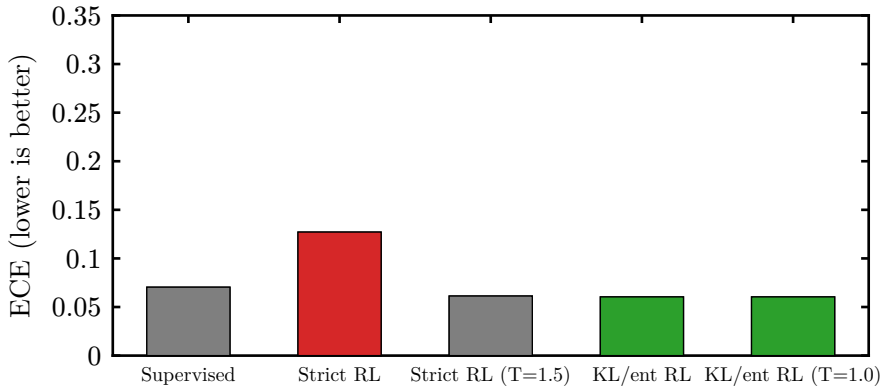


Figure 4: Expected calibration error by RL variant. Strict verifier RL degrades calibration (ECE 0.070 → 0.127) while leaving top-1 **unchanged at 9.13%** across all variants. Post-hoc temperature scaling needs $T = 1.5$ to recover; KL/entropy regularization fixes it at training time without that crutch.

Policy	T	Top-1	Top-10	NLL	ECE
Supervised	1.00	9.13%	78.01%	2.9170	0.0704
Strict verifier RL	1.00	9.13%	78.01%	3.0611	**0.1271**
Strict RL, temp-scaled	1.50	9.13%	78.01%	2.9063	0.0613
KL/entropy RL	1.00	9.13%	78.01%	2.9056	0.0604
KL/entropy RL, temp-scaled	1.00	9.13%	78.01%	2.9056	0.0604

Table 4: RL ablation, single cloned supervised checkpoint, default split.

The mechanism: strict policy gradient on a sparse exact reward signal pushes logit magnitudes without changing their relative ordering. The KL penalty bounds movement away from the reference policy; the entropy bonus discourages collapse to a single overconfident action. Together they reproduce the standard RLHF training stabilization, applied here to verifier rewards instead of preference rewards.

7.1. Reliability Diagram

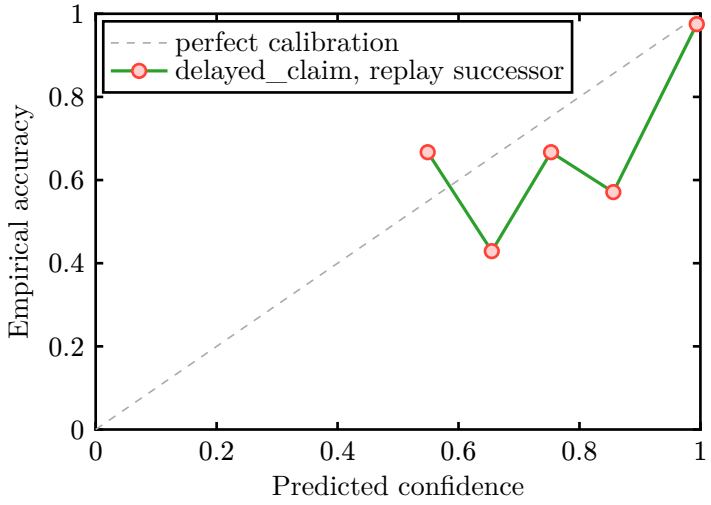


Figure 5: Reliability diagram for the trace-replay successor on `delayed_claim`. Bins from `results/latest/comparison.jsonl`. The high-confidence bin (894 of 1000 samples, $\hat{p} = 0.994$, $\hat{p}_{\text{emp}} = 0.975$) carries most of the mass; the low-population mid bins are noisier.

PLACEHOLDER: Replace with the prime-gap reliability diagram for supervised, strict-RL, and KL-RL once `make prime-gap` is run with `reliability_bins` capture enabled.

PLACEHOLDER — Required to elevate C3 from "true on prime-gap" to "a property of sparse-exact-reward RL":

- **E3.1** — Run strict-vs-regularized RL on `grid_path` and `delayed_claim` with the same protocol; report the analogous Table 4 for each.
- **E3.2** — Sweep the KL coefficient and entropy weight on a coarse grid for each task, plot the calibration-ranking Pareto frontier.
- **E3.3** — Per-bin reliability diagrams for **strict-RL vs KL-RL on prime-gap** (the current diagram uses delayed-claim as a stand-in because that's the only task whose `reliability_bins` are currently captured).
- **E3.4** — Confidence intervals across ≥ 5 seeds for the strict-vs-KL ECE delta.

8. Substrate

The hybrid architecture is the contribution; the substrate is the discipline that makes the claims auditable. Three pieces matter and would be a brief methods section in a longer paper:

- **Stable action IDs.** Trace rows carry `schema: "crlp.trace.v1"`, `trace_version: 1`, and `action_id_contract: "stable_unique_action_ids"`. Replay resolves `target_index` to the trace row's `action_id`, then maps that stable id back into the regenerated candidate row.
- **Checkpoint identity.** SHA-256 over the exact checkpoint bytes, plus `parent_model_hash`, `feature_encoder_version`, and `feature_set` in metadata. Successor models declare their parent; comparison reports carry both hashes and the evaluated `feature_set`.
- **Pure C, no third-party deps.** The neural library is dense layers, sigmoid/ReLU/tanh/linear, MSE and softmax cross-entropy, SGD, batched 4-by-4 tiled forward/backward, cloned networks, action scoring, pairwise ranking loss, and cached policy-gradient updates with optional KL/entropy regularization. ≈3k lines for the core; the prime-gap task is split into focused C modules for data, features, rule search, training, evaluation, and replay.

9. Limitations

L1. **Proposer learning not solved (Axis 2).** The fixed-budget baseline shows grammar order and a simple structural heuristic beat the current neural proposer on both the unconditional candidate grammar and the conditional gap grammar. The load-bearing claim is certificate-gated rule discovery, not learned proposer superiority.

L2. **Cross-task verifier-RL comparison missing (Axis 3).** The pipeline supports it; the comparison is not in canonical results yet.

L3. **Limited cross-domain gate coverage (Axis 1).** `grid_path` now has a budgeted legality-gate sanity check, but `delayed_claim` still lacks its certified gate implementation.

L4. **Statistical rigor.** Default `make prime-gap-experiments` runs 1 seed; the harness supports more. Deltas need confidence intervals across ≥ 5 seeds.

L5. **Learned-wheel calibration tradeoff** needs a reliability-diagram pass.

L6. **No literature comparison** on the calibration-collapse mechanism yet.

10. Conclusion

PLACEHOLDER — Write the conclusion last. Three sentences max: (a) certificate-gating is the architectural claim, (b) verifier-grounded rule synthesis is the constructive claim, (c) KL-regularized verifier RL is the training-stability claim. Avoid claiming the protocol scales; that is future work, not a result.

11. Roadmap to the ideal paper

This appendix tracks every experiment the current draft asks for but does not yet have. The goal is that every **PLACEHOLDER: ...** marker in the body resolves to one of these rows.

ID	Experiment	Effort	Defends
E1.1	Random-rule gate at matched verifier-call budget	2d	C1
E1.2	Extend `grid_path` beyond legality gate with admissible-heuristic pruning	3-4d	C1
E1.3	Investigate learned-wheel calibration tradeoff	1d	C1
E1.4	CIs across ≥ 5 seeds for 45.12 $\rightarrow$ 68.87 delta	0.5d	C1
E2.1	Fixed-budget proposer baselines across random seeds and budgets	1d	C2
E2.2	Conjunctive rule grammar extension	3-5d	C2
E2.3	Worked example of an accepted rule the hand-wheel misses	0.5d	C2
E3.1	Strict-vs-regularized RL on `grid_path` and `delayed_claim`	2-3d	C3
E3.2	KL/entropy Pareto sweep per task	2d	C3
E3.3	Reliability diagrams for strict-RL and KL-RL on prime-gap	0.5d	C3
E3.4	CIs across ≥ 5 seeds for strict-vs-KL ECE delta	0.5d	C3
W1	Related-work positioning paragraphs (see §2)	1d	all
W2	Conclusion + abstract revision after experiments	0.5d	all

Table 5: Roadmap. Total estimated effort: ≈3 weeks of focused work to take the current draft from "defensible as an arxiv preprint" to "submission-ready with all headline claims independently supported."

Minimum viable submission. E2.1, E1.3, E3.3, W1, W2. ≈4 days. Leaves Axes 2 and 3 with qualified scope but every claim in the body has a defensible experiment behind it.

Strong submission. Add E1.2, E2.3, E3.1, E1.4, E3.4. ≈2 weeks. All three axes have either a cross-task replication or a baseline strong enough to rule out the obvious alternative explanation.

Full ideal paper. Everything above. ≈3 weeks. The grammar extension (E2.2) is the optional ambition; if conjunctive rules generalize, the paper grows a fourth finding.